

C++ Programming Textbook Notes

Module: Destructors in C++ | Object Lifetime, Resource Cleanup & RAII

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. DESTRUCTORS IN C++: CONCEPTUAL FOUNDATIONS

What is a Destructor?

A **Destructor** is a special instance member function of a class whose primary responsibility is to execute cleanup operations immediately before an object is destroyed and its memory reclaimed by the operating system.

Why Do We Need Destructors? (The Resource Leak Problem)

When an object is created on the stack, the memory occupied by its direct members is automatically reclaimed when the function or block scope exits. However, if an object acquires external resources during its lifetime—such as **dynamic heap memory** (via `new`), open file handles, database connections, or network sockets—simply popping the object off the stack leaves those external resources stranded in RAM!

If an object contains a pointer pointing to heap memory, destroying the object without deleting that heap memory causes a severe **Memory Leak**. The address is lost forever, yet the RAM remains occupied. A destructor guarantees that these acquired resources are safely deallocated right before the object disappears.

[THE RESOURCE LEAK PROBLEM WITHOUT A DESTRUCTOR]

1. Object with Pointer on Stack:

```
Object 'obj' (Stack Memory)
  int *ptr = 0x8000 (Heap Address)  → points to → [ Dynamic Heap Memory: 0x8000 ]
```

↓
▼ Scope Exits!

2. If NO Destructor is Defined:

Stack Memory Reclaimed!
[DEALLOCATED]

Dangling Heap Memory (Memory Leak!):
[0x8000: Stranded RAM, Cannot be Freed!]

3. With a Destructor (~ClassName()):

Destructor executes: 'delete ptr;' → Deallocates 0x8000 on Heap!
Stack Memory Reclaimed Cleanly! → Zero Memory Leaks!

2. SYNTAX & ESSENTIAL RULES FOR DESTRUCTORS

Core Characteristics & Constraints

- **Name Matching with Tilde (~):** The destructor's name is identical to the class name, but must be preceded by a **tilde** (~) symbol (e.g., `~Complex()`).
- **No Return Type:** Destructors have **no return type** (not even `void`) and cannot return any value.
- **Takes Zero Arguments:** Destructors accept **no arguments/parameters**. Consequently, **destructors cannot be overloaded!** There is only ever one destructor per class.
- **Cannot Be Static:** Destructors can **never be static** because they must operate on a specific object instance bound to the `this` pointer.

- **Implicit Invocation:** You do not call a destructor explicitly in standard code. It is **implicitly invoked** by the compiler right when an object is about to be destroyed (when leaving scope, or when delete is called on a heap pointer).

Destructor Syntax

```
class ClassName {
public:
    // Constructor
    ClassName() { /* acquire resources / initialize */ }

    // Destructor
    ~ClassName() {
        // Release resources, delete heap pointers, close files
    }
};
```

3. CONSTRUCTOR VS. DESTRUCTOR COMPARISON

Feature	Constructor	Destructor
Purpose	Initializes object state and allocates resources.	Cleans up and releases acquired resources.
Invocation Instant	Invoked immediately when an object is created .	Invoked immediately before an object is destroyed .
Naming Convention	Identical to Class Name: ClassName ()	Preceded by tilde: ~ClassName ()
Parameters / Overloading	Can take parameters and can be overloaded (Default, Parameterized, Copy).	Takes no parameters. Overloading is strictly impossible .
Count Per Class	Multiple constructors allowed via overloading.	Exactly one destructor per class.

4. EXECUTION FLOW: ORDER OF DESTRUCTION IN MEMORY

The LIFO Destruction Rule

When multiple objects are created in the same block scope on the Call Stack, they are initialized in order of declaration. When that block scope exits, the compiler destroys the objects in **reverse order of creation (Last-In, First-Out / LIFO)**.

```
[ LIFO OBJECT DESTRUCTION FLOW ]

Creation Order:    Object 1 (First)  →  Object 2  →  Object 3 (Last)
Destruction Order: Object 3 (~Obj3)  →  Object 2 (~Obj2) →  Object 1 (~Obj1)
```

5. PRACTICAL CODE DEMONSTRATIONS

Example 1: Basic Destructor Invocation via Function Scope

```
// Basic Destructor Demonstration in C++
#include <iostream>
using namespace std;

class Complex {
private:
    int a, b;

public:
    // Constructor
    Complex() {
        a = 0;
        b = 0;
        cout << "Constructor executed for Complex object." << endl;
    }

    // Destructor: Invoked implicitly when object is going to be destroyed
    ~Complex() {
        cout << "Destructor executed! Object destroyed." << endl;
    }
};

void fun() {
    cout << "Inside fun(): Creating local object 'obj'..." << endl;
    Complex obj; // Constructor runs here
    cout << "Exiting fun() block..." << endl;
} // 'obj' goes out of scope here; Destructor runs automatically!

int main() {
    cout << "Calling fun() from main()..." << endl;
    fun();
    cout << "Returned to main(). Execution completed." << endl;
    return 0;
}
```

Example 2: Real-World Resource Management (Preventing Memory Leaks)

```
// Real-World RAII Destructor Demonstration: Deallocating Dynamic Heap Memory
#include <iostream>
using namespace std;

class DynamicArray {
private:
    int* data;
    int size;

public:
    // Constructor allocates dynamic heap memory
    DynamicArray(int s) {
        size = s;
        data = new int[size];
        cout << "Allocated dynamic heap memory for " << size << " integers." << endl;
    }

    // Destructor safely frees the allocated heap memory to avoid memory leaks
    ~DynamicArray() {
        delete[] data;
        cout << "Destructor freed dynamic heap memory successfully." << endl;
    }
};

int main() {
    cout << "Creating DynamicArray instance in main..." << endl;
    {
        DynamicArray arr(50); // Heap memory allocated
        cout << "Working with array inside inner block..." << endl;
    } // Inner scope ends -> Destructor automatically invoked -> Heap freed!

    cout << "Continuing main() execution without memory leaks!" << endl;
    return 0;
}
```

Common Gotcha: Explicit Destructor Invocations

Never call a destructor explicitly on a normal stack object (e.g., writing `obj.~Complex()`). The compiler will still call the destructor automatically when the object exits its scope, resulting in a disastrous **Double Free Error!**

Best Practice: RAII (Resource Acquisition Is Initialization)

Bind resource acquisition to constructor execution and resource release to destructor execution. Destructors ensure that file streams, mutex locks, and dynamic memory are always cleaned up, even if an exception is thrown!